



Universidad Autónoma de Campeche
Ingeniería en Tecnología de Software

Materia

CONCURRENCIA Y PARALELISMO

Maestro

Sergio A Noh Puch

Actividad

Condiciones de carrera y sección crítica

Alumna

Gladys Candy Chumá Pérez

Matrícula

70238

Introducción

En esta práctica se trabajó con hilos para entender un poco mejor qué pasa cuando varios hilos utilizan una misma variable al mismo tiempo. También se probó qué pasa cuando no se utiliza sincronización y después se utilizó un mutex para evitar problemas entre los hilos.

Objetivo

Implementar un programa utilizando hilos para observar las condiciones de carrera y comparar su funcionamiento con y sin sincronización mediante mutex.

Ejecución sin sincronización

Primero se implementaron dos hilos que modifican una variable compartida llamada **globa_counter**, la cual comienza con un valor de 20. Un hilo se encarga de incrementar el contador 1000 veces y el otro de disminuirlo la misma cantidad de veces.

Al ejecutar el programa sin utilizar un mutex se pudieron observar algunos problemas en la salida. Aunque el contador terminó en 20, los mensajes que se escribieron en el archivo no siempre quedaron de la manera esperada. También se pudieron observar mensajes mezclados.

```
1 New line
2
3 Gladys...
4 Gladys...
5 New line
6
7 New line
8
9 New line
10
11 New line
12
13 Gladys...
14 Gladys...
15 Gladys...
16 Gladys...New line
17 New line
18
19 New line
```

Ejecución con sincronización

Después se agregó `pthread_mutex` para controlar el acceso de los hilos a la variable compartida y también a la escritura del archivo.

El mutex hace que un hilo tenga que esperar mientras el otro está utilizando la parte del código que está protegida. De esta manera se evita que los dos hilos intenten modificar o escribir al mismo tiempo.

```
989     New line
990     New line
991     New line
992     New line
993     New line
994     New line
995     New line
996     New line
997     New line
998     New line
999     New line
1000    New line
1001    Gladys...
1002    Gladys...
1003    Gladys...
1004    Gladys...
1005    Gladys...
1006    Gladys...
1007    Gladys...
1008    Gladys...
```

Conceptos

Condición de carrera

Una condición de carrera ocurre cuando dos o más hilos utilizan un mismo recurso al mismo tiempo y no existe una sincronización adecuada. Esto puede hacer que los hilos interfieran entre ellos y se pierdan algunas actualizaciones.

Sección crítica

Una sección crítica es una parte del código donde se utiliza o modifica un recurso que es compartido entre varios hilos.

En este caso, las operaciones `global_counter++` y `global_counter--` utilizan la misma variable, por lo que es importante proteger esta parte del código. El mutex se utiliza justamente para evitar que los hilos entren al mismo tiempo en esa parte.

Código ensamblador

Para esta parte se utilizó el comando:

`gcc -S race_test.c`

Esto permitió obtener el código ensamblador del programa y observar cómo se realizan algunas de las operaciones del contador.

```
51      .L3:
52      movl    global_counter(%rip), %eax
53      addl    $1, %eax
54      movl    %eax, global_counter(%rip)

114     .L8:
115     movl    global_counter(%rip), %eax
116     subl    $1, %eax
117     movl    %eax, global_counter(%rip)
```

Función de las instrucciones

- **movl**: Se utiliza para mover o copiar un valor, por ejemplo, entre la memoria y un registro.
- **addl**: Se utiliza para restar un valor. En este caso resta 1 al contador.
- **subl**: Se utiliza para restar un valor. En este caso resta 1 al contador.

Con esto se puede ver que una operación como aumentar o disminuir una variable no necesariamente se hace en una sola instrucción. Primero se obtiene el valor, después se realiza la operación y finalmente se guarda el nuevo valor.

Por eso, cuando varios hilos trabajan con la misma variable sin algún mecanismo de sincronización, pueden interferir entre ellos.

Conclusión

En esta práctica se pudo observar la diferencia entre trabajar con varios hilos sin sincronización y hacerlo utilizando un mutex. Aunque el contador terminó en 20 en las dos pruebas, sí hubo diferencias al momento de escribir en el archivo `readme.txt`.

Sin el mutex se presentaron mensajes mezclados y no se obtuvieron todas las líneas esperadas. Después de agregar la sincronización, se pudieron escribir las 2000 líneas de una manera más ordenada.

También, al revisar el código ensamblador, se pudo entender un poco mejor cómo se realizan las operaciones de incremento y decremento y por qué es importante proteger las partes del código donde se utilizan recursos compartidos.